DA3408 Assignment 3: Spark vs. Ray — The Data Engineering Duel

AIOps Assignment 3

October 2026

Abstract

This report compares Apache Spark and Ray Data on an identical preprocessing pipeline over 36 monthly NYC Yellow Taxi inputs. The pipeline performs ingestion, cleansing, duplicate removal, timestamp formatting, a taxi-zone lookup join, average-speed derivation, and Parquet export. Native and Python-UDF variants were measured end-to-end. The recorded wall-clock times were 392.50 s and 432.34 s for Spark, and 636.96 s and 711.03 s for Ray. Resource logs were also analyzed for peak CPU and RAM usage.

1 Objective

The assignment requires an identical data preprocessing pipeline in Spark and Ray, deployed as distributed clusters and benchmarked on a large-scale NYC Taxi dataset. Required measurements are total wall-clock time from ingestion through export, peak CPU and memory usage, and Python-UDF overhead. The transformation includes a heavy join with taxi-zone information and an average-speed feature.

2 System and Pipeline

The experiment used an Ubuntu 24.04.4 LTS ARM64 VM with Python 3.10.13, Java 17, Spark 4.2.0, Ray 2.20.0 and PyArrow 15.0.2. Each framework was configured with two workers. Spark used a master on port 7077 with its UI on port 8080; Ray used a head node on port 6379 with its dashboard on port 8265.

Both implementations process the same 36 monthly Yellow Taxi files. The common logical pipeline is:

1. read the monthly Parquet input;
2. normalize the TLC fields and timestamps;
3. remove null/invalid records and duplicates;
4. join `PULocationID` with `taxi_zone_lookup.csv`;
5. calculate `average_speed_mph`;
6. add pickup date and export the structured result to Parquet.

Spark uses SQL expressions for the native speed calculation and a Python UDF in UDF mode. Ray provides equivalent native and Python-based transformation modes.

3 Benchmark Method

Each monthly file is processed sequentially while a single end-to-end timer covers ingestion through export. This avoids treating the 36 months as 36 independent benchmarks. The supplied metric JSON files contain the recorded monthly timings and total wall-clock times. CPU and memory were recorded with VM-level `top` and `free -m` samples.

For CPU, each sample is converted to utilization using $100 - \text{idle}$. Peak RAM is the maximum `used` value in the corresponding `free -m` samples.

4 End-to-End Results

Table 1: Recorded end-to-end wall-clock results.

Framework	Mode	Time (s)	Time (min)
Spark	Native	392.50	6.54
Spark	UDF	432.34	7.21
Ray	Native	636.96	10.62
Ray	UDF	711.03	11.85

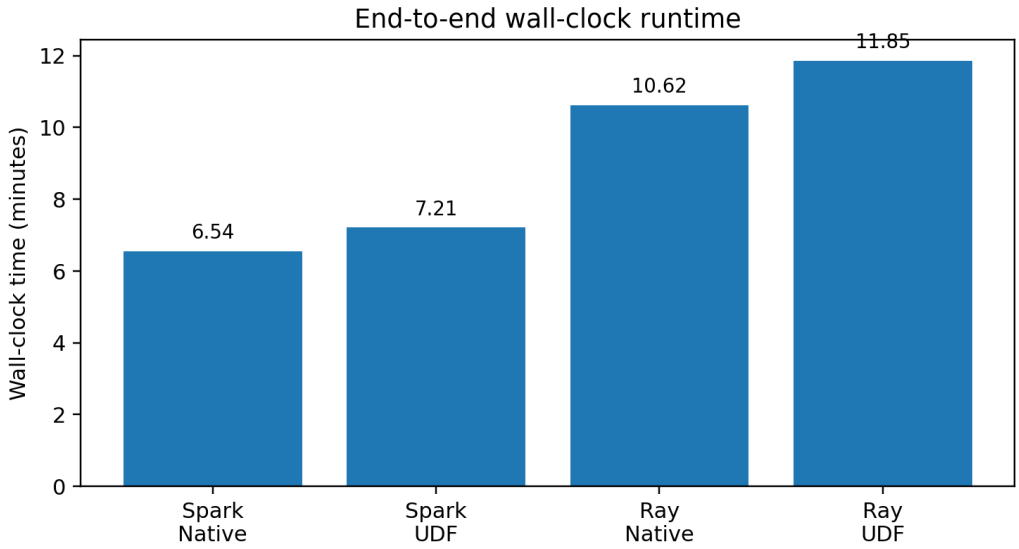


Figure 1: End-to-end wall-clock runtime from ingestion through Parquet export.

Spark is faster than Ray in both measured modes on the tested VM. Spark Native completes in 392.50 s, compared with 636.96 s for Ray Native. The UDF variants take 432.34 s and 711.03 s. These results describe the tested configuration and should not be interpreted as a universal framework ranking.

5 Python UDF Overhead

For each framework:

$$\text{overhead} = \frac{T_{UDF} - T_{Native}}{T_{Native}} \times 100.$$

Spark:

$$\frac{432.3402 - 392.5020}{392.5020} \times 100 = \mathbf{10.15}\%.$$

Ray:

$$\frac{711.0328 - 636.9566}{636.9566} \times 100 = \mathbf{11.63}\%.$$

Thus the Python UDF increased total runtime by 10.15% in Spark and 11.63% in Ray.

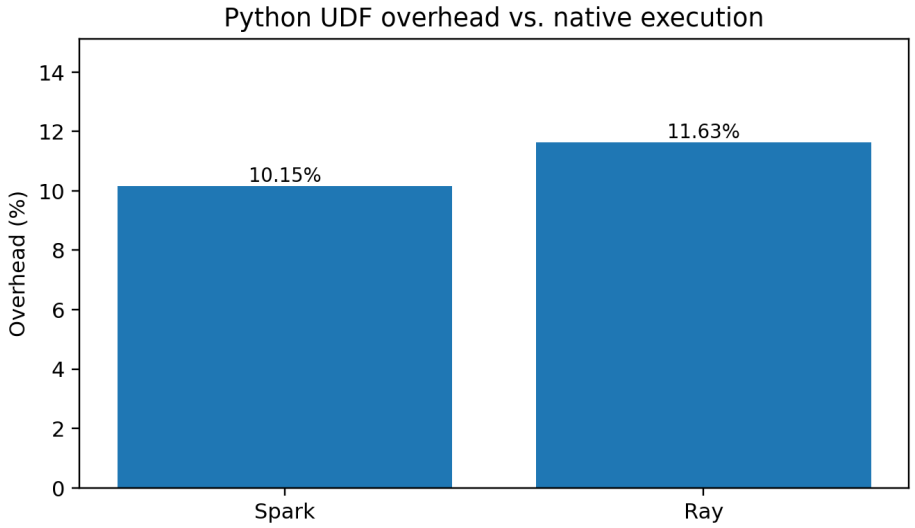


Figure 2: Relative Python-UDF overhead.

6 Resource Utilization

Table 2: Peak resource usage from recorded monitoring samples.

Framework	Mode	Peak CPU (%)	Peak RAM (GiB)	P95 CPU (%)
Spark	Native	100.0	7.46	95.7
Spark	UDF	100.0	6.73	100.0
Ray	Native	100.0	5.56	98.3
Ray	UDF	100.0	5.58	97.9

All four runs reached an instantaneous CPU peak of 100%. Mean CPU utilization was 65.9% for Spark Native, 68.3% for Spark UDF, 76.6% for Ray Native, and 72.8% for Ray UDF. Peak RAM was 6.88, 6.10, 4.71, and 4.83 GiB, respectively.

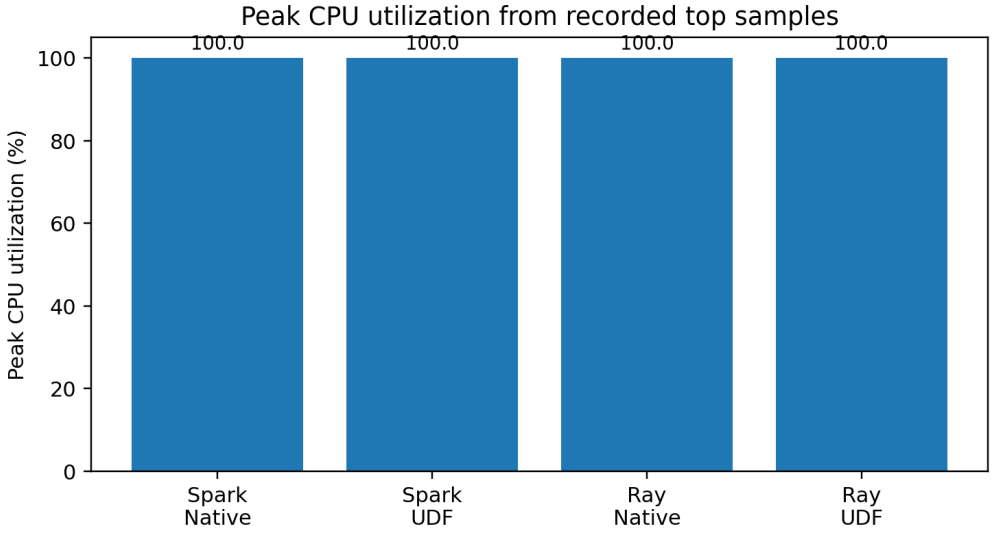


Figure 3: Peak CPU utilization from VM-level monitoring samples.

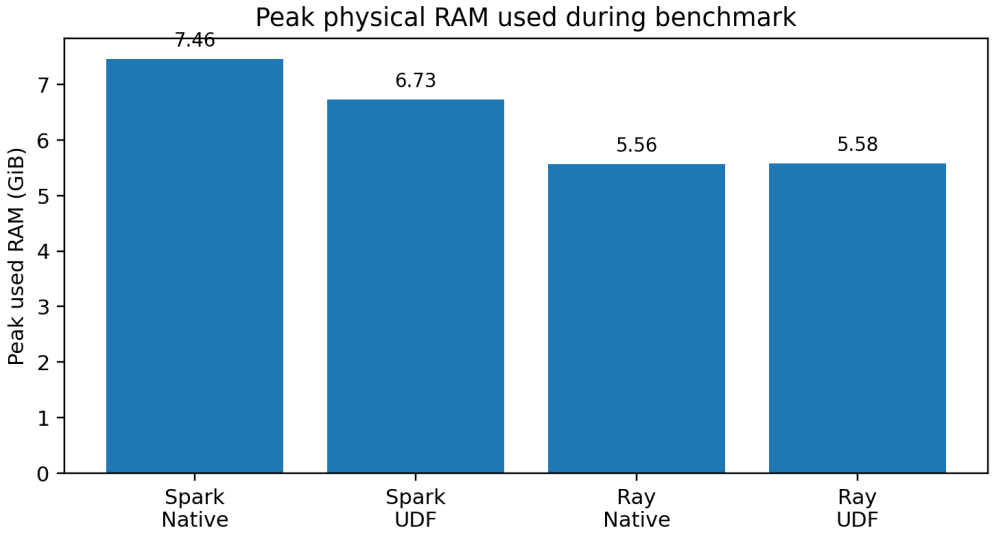


Figure 4: Peak physical RAM used during each benchmark.

7 Monthly Performance

The monthly timings are retained at their original granularity rather than being replaced with synthetic values. This exposes workload variation across all 36 monthly inputs.

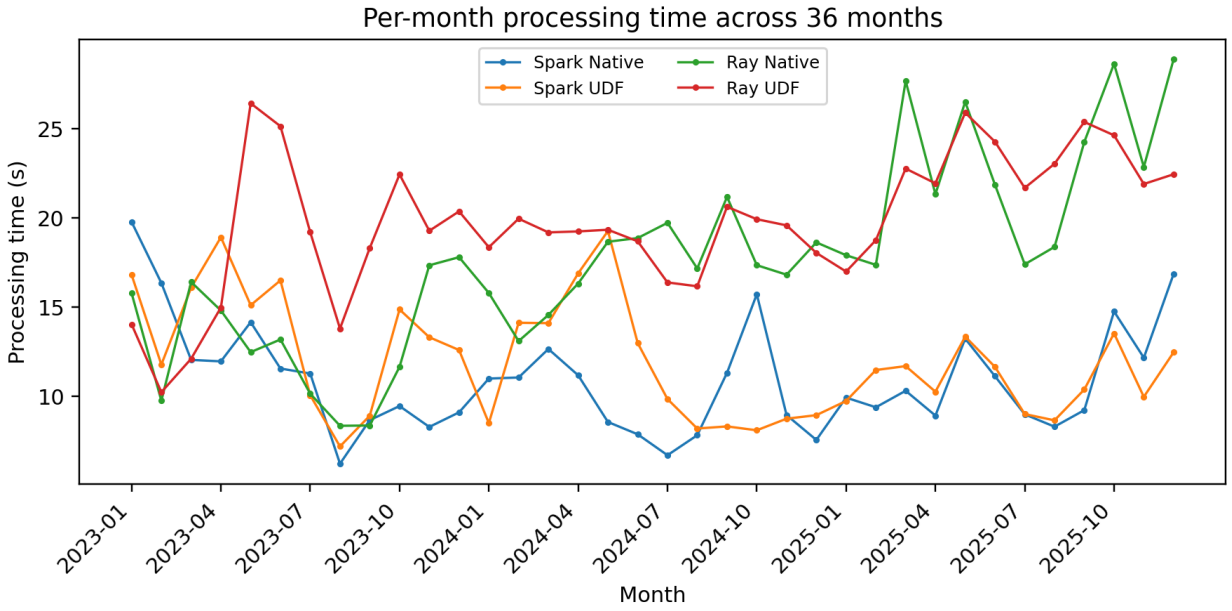


Figure 5: Recorded processing time for each monthly input.

8 Correctness and Performance Tuning

The final Ray implementation was made parity-focused and memory-safe. It performs month-level exact deduplication to match Spark's `dropDuplicates()` semantics, computes duration from timestamps floored to whole seconds, preserves nullable integer columns, handles taxi-zone lookup strings consistently with Spark, fixes string column types, and uses the same average-speed expression.

The Ray implementation reads only required Parquet columns, bounds map-batch size, splits input into multiple blocks, caps Ray Data object-store usage, and controls output block size. This was necessary because an earlier global all-to-all deduplication strategy caused memory pressure. The final implementation therefore optimizes memory without changing the intended pipeline semantics.

The completed main parity comparison established matching 36-month row counts of 127,627,157 and a detailed

January logical-value check with no row or formula mismatches. During the final UDF parity verification, 18 consecutive monthly outputs passed the rigorous logical-row comparison before the exhaustive check was stopped to avoid an unnecessary additional long hash pass.

9 Performance Discussion

The measured results show lower absolute wall-clock time for Spark on the tested VM. Spark's structured execution path is well suited to the tabular preprocessing workload, including the lookup join and expression-based transformation. Ray's Python-native data processing model provides flexibility for Python-centric workflows, but its end-to-end runtime was higher in this experiment.

The UDF comparison is more nuanced: Spark's UDF overhead was 10.15%, while Ray's was 11.63%. This measures the cost of replacing a native expression with a Python-level transformation within each complete pipeline, not an isolated microbenchmark of a language boundary.

From a BI-first perspective, Spark's structured execution and mature dataframe optimization are strong fits for conventional ETL. From an AI-first perspective, Ray's Python-native execution model is attractive when preprocessing is tightly coupled to custom Python or ML workloads. The experiment demonstrates a workload- and configuration-dependent trade-off rather than a universal winner.

10 Conclusion

Under the recorded configuration, Spark completed the native pipeline in 392.50 s and the UDF pipeline in 432.34 s. Ray completed the corresponding runs in 636.96 s and 711.03 s. UDF overhead was 10.15% for Spark and 11.63% for Ray. The final Ray implementation achieved the required memory-safe execution strategy while maintaining the tested parity properties.

A AI Usage Appendix

AI assistance was used within the assignment's stated fair-use boundary. It was used for:

- translating/adapting equivalent preprocessing logic between PySpark and Ray Data;
- diagnosing Ray memory pressure and explaining the unsuccessful global-deduplication approach;
- developing memory-safe batching, block sizing, and month-level deduplication strategies;
- constructing parity-check and benchmark-analysis scripts;
- organizing recorded results and producing report documentation and plots.

Cluster configuration and network setup were performed manually. The benchmark values in this report were extracted from the supplied execution metric files and resource-monitoring logs; no synthetic benchmark values were introduced. The README documents how to obtain the omitted dataset and reproduce the four pipeline runs.